**K. J. Somaiya School of Engineering, Mumbai-77**

| |
|---|
| **Batch: A2 Roll No.: 16010324044** |
| **Experiment / assignment / tutorial No. 2** |
| **Grade: AA / AB / BB / BC / CC / CD /DD** |
| **Signature of the Staff In-charge with date** |

**TITLE:** Shell programming using Unix/Linux
**Objectives:** To write simple shell scripts using shell programming fundamentals
**OUTCOME:** Student will be able to perform basic operating system tasks using command-line and will be able to write shell programming

The activities of a shell are not restricted to command interpretation alone. The shell also has rudimentary programming features. Shell programs are stored in a file (with extension .**sh**). Shell programs run in interpretive mode. The original UNIX came with the Bourne shell (**sh**) and it is universal even today. C shell (**csh**) and Korn shell (**ksh**) are also widely used. Linux offers Bash shell (**bash**) as a superior alternative to Bourne shell.

**Preliminaries**

1. Comments in shell script start with #.
2. Shell variables are loosely typed i.e. not declared. Variables in an expression or output must be prefixed by **$**.
3. The **read** statement is shell's internal tool for making scripts interactive.
4. Output is displayed using **echo** statement.
5. Expressions are computed using the **expr** command. Arithmetic operators are + -

* / %. Meta characters *** ( )** should be escaped with a \.
6. The shell scripts are executed

**$** sh *filename*

Decision-making

Shell supports decision-making using **if** statement. The **if** statement like its counterpart programming languages has the following formats.

The set of relational operators are –eq –ne –gt –ge –lt –le and logical operators used in conditional expression are –a –o !

**Multi-way branching**

The case statement is used to compare a variables value against a set of constants. If it matches a constant, then the set of statements followed after ) is executed till a ;; is encountered. The optional *default* block is indicated by *****. Multiple constants can be specified in a single pattern separated by |.

Case *variable* in
*Constant 1*)
*statements* ;;
*constant 2*)
*statements* ;;
. . .
*)
*statements*
case

**Loops**
Shell supports a set of loops such as **for**, **while** and **until** to execute a set of statements repeatedly. The body of the loop is contained between **do** and **done** statement.

The **for** loop doesn't test a condition, but uses a list instead.

for *variable* in *list*
do
*statements*
done

The **while** loop executes the *statements* as long as the condition remains true.

while [ *condition* ] do
*statements*
done

The **until** loop complements the while construct in the sense that the *statements* are executed as long as the condition remains false.

until [ *condition* ] do
*statements*
done

1) **Write a shell program to reverse a string.**

```
dmspe@D-Laptop:~$ echo "Enter a string:"
read str

len=${#str}
rev=""

while [ $len -gt 0 ]
do
    rev=$rev${str:$len-1:1}
    len=$((len-1))
done

echo "Reversed String: $rev"
Enter a string:
loop
Reversed String: pool
```

2) **Write a shell program to find whether number is prime or not.**

```
dmspe@D-Laptop:~$ echo "Enter a number:"
read n

i=2
flag=0

while [ $i -lt $n ]
do
    if [ $((n%i)) -eq 0 ]
    then
        flag=1
        break
    fi
    i=$((i+1))
done

if [ $flag -eq 0 ]
then
    echo "$n is Prime"
else
    echo "$n is Not Prime"
fi
Enter a number:
11
11 is Prime
```

3) **Write a shell program to find Fibonacci series.**

```
dmspe@D-Laptop:~$ echo "Enter number of terms:"
read n

a=0
b=1

echo "Fibonacci Series:"

i=0
while [ $i -lt $n ]
do
    echo -n "$a "
    temp=$((a+b))
    a=$b
    b=$temp
    i=$((i+1))
done

echo
Enter number of terms:
7
Fibonacci Series:
0 1 1 2 3 5 8
```

4) **Write a shell program for Calculator design which will do operations as addition, division, multiplication, logarithm function.**

```
dmspe@D-Laptop:~$ echo "Enter first number:"
read a

echo "Enter second number:"
read b

echo "Choose Operation"
echo "1. Addition"
echo "2. Division"
echo "3. Multiplication"
echo "4. Logarithm"

read ch

case $ch in

1)
echo "Addition = $((a+b))"
;;

2)
echo "Division = $(echo "scale=2; $a/$b" | bc)"
;;

3)
echo "Multiplication = $((a*b))"
;;

4)
echo "Natural Log of First Number = $(echo "l($a)" | bc -l
;;

*)
echo "Invalid Choice"
;;

esac
```

```
Enter first number:
20
Enter second number:
4
Choose Operation
1. Addition
2. Division
3. Multiplication
4. Logarithm
2
Division = 5.00
```

## Conclusion

In this experiment, I successfully wrote and executed shell programs for reversing a string, checking whether a number is prime, generating the Fibonacci series, and performing calculator operations using Bash shell scripting. The experiment demonstrated the use of variables, input/output, decision-making statements, loops, and the case statement in Linux shell programming.

**Signature of faculty in-charge**